

CS5340 - Uncertainty Modelling in AI

Lecture 4: Variable Elimination and Belief Propagation

Scribed by: Claude (Anthropic) | Curated By: Assoc. Prof. Lee Gim Hee
Semester 1, AY2026/27 — National University of Singapore

Disclaimer: This scribe was generated by **Claude (Anthropic)** and curated by **Assoc. Prof. Lee Gim Hee**. It is a **supplementary revision aid** that summarizes the lecture slides, which remain the **primary learning material** prepared by the lecturer. Despite curation, the scribe may still contain errors, omissions, or simplifications. Please use it only alongside the official slides, textbook references, and your own lecture notes.

1 Lecture Overview

- **Motivation:** Lectures 2 and 3 showed how directed (Bayesian networks) and undirected (MRFs) graphical models compactly represent a joint distribution. We now ask the central computational question: **given such a model, how do we efficiently compute conditional and marginal probabilities?**
- **Goal:** replace the intractable $O(k^n)$ naive summation with algorithms that **exploit the factorization** encoded by the graph.
- **Roadmap:**
 1. Formulate the general **probabilistic inference problem** $p(x_F | x_E)$.
 2. Derive the **Variable Elimination** algorithm using the distributive law and **evidence potentials**.
 3. Analyze its **computational complexity** using the **reconstituted graph**, elimination cliques and treewidth.
 4. Specialize elimination to **trees**, where intermediate factors become **messages**.
 5. Present the **sum-product (belief propagation)** algorithm, which computes **all** single-node marginals in one run.

Learning Outcomes

By the end of this lecture, you should be able to:

1. Use the Variable Elimination algorithm to compute the conditional probability of a single random variable X_f , i.e. $p(x_f | x_E)$.
2. Explain the computational complexity of variable elimination using the reconstituted graph.
3. Use the sum-product algorithm to compute all single-node marginals for “tree-like” graphical models.

2 The Probabilistic Inference Problem

Definition : Probabilistic Inference Problem

Let E and F be **disjoint** subsets of the node indices of a graphical model, so X_E and X_F are disjoint subsets of the random variables.

- X_E : the **evidence nodes** (observed) on which we condition.
- X_F : the **query nodes** whose conditional distribution we want.
- $X_R = X_V \setminus (X_E, X_F)$: the **nuisance variables**, which must be marginalized out; $\{X_E, X_R, X_F\}$ are all the random variables in the model.

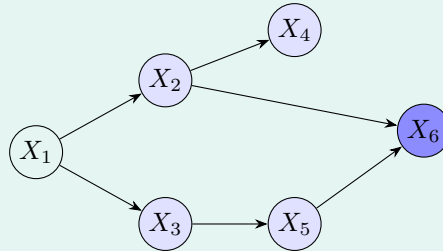
Goal: compute $p(x_F | x_E)$ for **arbitrary** subsets E and F . This is the general inference problem for graphical models, **directed or undirected**.

By the definition of conditional probability and marginalization of the joint,

$$p(x_F | x_E) = \frac{p(x_E, x_F)}{p(x_E)}, \quad p(x_E, x_F) = \sum_{x_R} p(x_E, x_F, x_R), \quad p(x_E) = \sum_{x_F} p(x_E, x_F).$$

Worked Example: The Running Example

Throughout the lecture we use the six-node DGM below. **Dark** shading marks the evidence node $X_E = \{X_6\}$, the **unshaded** node is the query node $X_F = \{X_1\}$, and the **lightly shaded** nodes $X_R = \{X_2, \dots, X_5\}$ must be marginalized out.



The joint distribution factorizes according to the graph as

$$p(x_1, \dots, x_6) = p(x_1) p(x_2 | x_1) p(x_3 | x_1) p(x_4 | x_2) p(x_5 | x_3) p(x_6 | x_2, x_5).$$

(Image source: M. I. Jordan, “An Introduction to Probabilistic Graphical Models”, 2002.)

Key Rule: Naive Summation Is Intractable

$\sum_{x_R}$ expands into a **sequence of summations**, one for each random variable indexed by R . Summing directly over the joint table of n variables that each take k states costs

$$O(k^n).$$

E.g. $p(x_1, \dots, x_5) = \sum_{x_6} p(x_1, \dots, x_6)$ already touches a joint table of size k^6 , i.e. $O(k^6)$ operations for a **single** sum.

3 Variable Elimination

We first look at computing the conditional probability of a **single** query node X_F given an **arbitrary** set of evidence nodes X_E .

Definition : Variable Elimination Algorithm

An efficient inference algorithm that exploits (i) the **factorization** (i.e. the conditional independences) of the graphical model and (ii) the **distributive law** of multiplication over addition, summing out the nuisance variables **one at a time**.

3.1 Exploiting the Factored Form

Key Rule: Push Sums Inward with the Distributive Law

Write the joint in its factored form and move each sum past every factor that does not depend on the summed variable:

$$\begin{aligned} p(x_1, \dots, x_5) &= \sum_{x_6} p(x_1)p(x_2 | x_1)p(x_3 | x_1)p(x_4 | x_2)p(x_5 | x_3)p(x_6 | x_2, x_5) \\ &= p(x_1)p(x_2 | x_1)p(x_3 | x_1)p(x_4 | x_2)p(x_5 | x_3) \underbrace{\sum_{x_6} p(x_6 | x_2, x_5)}_{\text{table of size } k^3}. \end{aligned}$$

The single sum drops from $O(k^6)$ to $O(k^3)$ operations. In general we get $O(k^r)$ instead of $O(k^n)$ complexity, where $r \ll n$ is the size of the largest table created (made precise in Section 4).

Remark: without evidence, $\sum_{x_6} p(x_6 | x_2, x_5) = 1$; the point of the example is the **size** of the table each sum touches. Once X_6 is observed (below), this factor is no longer trivially one.

3.2 Evidence Nodes

Consider $p(x_1 | x_6) = p(x_1, x_6)/p(x_6)$, where X_6 is the evidence node and X_1 the query node. Since X_6 is **observed**, it is a **fixed constant** that does not contribute to the computational complexity. We denote an observed evidence node by $\bar{X}_i$:

$$p(x_1 | \bar{x}_6) = \frac{p(x_1, \bar{x}_6)}{p(\bar{x}_6)}.$$

Worked Example: Observing Evidence Slices a Table

Let $x_i \in \{0, 1\}$ and suppose we observe $\bar{X}_6 = 1$. Conditioning simply keeps the rows of $p(x_6 | x_2, x_5)$ consistent with the observation (in red):

X_2	X_5	X_6	$p(x_6 \mid x_2, x_5)$		X_2	X_5	$p(\bar{x}_6=1 \mid x_2, x_5)$
0	0	0	v_0		0	0	v_1
0	0	1	v_1		0	1	v_3
0	1	0	v_2	$\xrightarrow{\bar{X}_6=1}$	1	0	v_5
0	1	1	v_3		1	1	v_7
1	0	0	v_4				
1	0	1	v_5				
1	1	0	v_6				
1	1	1	v_7				

We are taking a **2D slice of the 3D** table of probabilities (or potentials).

3.3 Eliminating One Variable at a Time

Worked Example: Computing $p(x_1 \mid \bar{x}_6)$ by Hand

Push each sum as far right as possible, then evaluate from the inside out:

$$\begin{aligned}
 p(x_1, \bar{x}_6) &= \sum_{x_2} \sum_{x_3} \sum_{x_4} \sum_{x_5} p(x_1) p(x_2 \mid x_1) p(x_3 \mid x_1) p(x_4 \mid x_2) p(x_5 \mid x_3) p(\bar{x}_6 \mid x_2, x_5) \\
 &= p(x_1) \sum_{x_2} p(x_2 \mid x_1) \sum_{x_3} p(x_3 \mid x_1) \sum_{x_4} p(x_4 \mid x_2) \underbrace{\sum_{x_5} p(x_5 \mid x_3) p(\bar{x}_6 \mid x_2, x_5)}_{m_5(x_2, x_3)} \\
 &= p(x_1) \sum_{x_2} p(x_2 \mid x_1) \sum_{x_3} p(x_3 \mid x_1) m_5(x_2, x_3) \underbrace{\sum_{x_4} p(x_4 \mid x_2)}_{m_4(x_2)} \\
 &= p(x_1) \sum_{x_2} p(x_2 \mid x_1) m_4(x_2) \underbrace{\sum_{x_3} p(x_3 \mid x_1) m_5(x_2, x_3)}_{m_3(x_1, x_2)} \\
 &= p(x_1) \underbrace{\sum_{x_2} p(x_2 \mid x_1) m_4(x_2) m_3(x_1, x_2)}_{m_2(x_1)} \\
 &= p(x_1) m_2(x_1).
 \end{aligned}$$

Line by line we eliminate X_5 , then X_4 (possible because m_5 is independent of X_4), then X_3 (since m_4 is independent of X_3), and finally X_2 . An additional sum over X_1 gives the evidence probability, and dividing gives the desired conditional:

$$p(\bar{x}_6) = \sum_{x_1} p(x_1) m_2(x_1), \quad p(x_1 \mid \bar{x}_6) = \frac{p(x_1) m_2(x_1)}{\sum_{x_1} p(x_1) m_2(x_1)}.$$

Definition : Intermediate Factor $m_i(x_{S_i})$

$m_i(x_{S_i})$ denotes the expression that results from performing $\sum_{x_i}$, where X_{S_i} are the variables, **other than** X_i , that appear in the summand. Summands can be pushed in because of the

distributive law.

Worked Example: Marginalization Table for $m_5(x_2, x_3)$

Let $x_i \in \{0, 1\}$ and $\bar{X}_6 = 1$, with

X_2	X_5	$p(\bar{x}_6=1 \mid x_2, x_5)$	X_3	X_5	$p(x_5 \mid x_3)$
0	0	a_1	0	0	b_1
0	1	a_2	0	1	b_2
1	0	a_3	1	0	b_3
1	1	a_4	1	1	b_4

Then $m_5(x_2, x_3) = \sum_{x_5} p(x_5 \mid x_3) p(\bar{x}_6=1 \mid x_2, x_5)$ is the table

X_2	X_3	$m_5(x_2, x_3)$
0	0	$p(x_5=0 \mid x_3=0) p(\bar{x}_6=1 \mid x_2=0, x_5=0) + p(x_5=1 \mid x_3=0) p(\bar{x}_6=1 \mid x_2=0, x_5=1) = b_1 a_1 + b_2 a_2$
0	1	$p(x_5=0 \mid x_3=1) p(\bar{x}_6=1 \mid x_2=0, x_5=0) + p(x_5=1 \mid x_3=1) p(\bar{x}_6=1 \mid x_2=0, x_5=1) = b_3 a_1 + b_4 a_2$
1	0	$p(x_5=0 \mid x_3=0) p(\bar{x}_6=1 \mid x_2=1, x_5=0) + p(x_5=1 \mid x_3=0) p(\bar{x}_6=1 \mid x_2=1, x_5=1) = b_1 a_3 + b_2 a_4$
1	1	$p(x_5=0 \mid x_3=1) p(\bar{x}_6=1 \mid x_2=1, x_5=0) + p(x_5=1 \mid x_3=1) p(\bar{x}_6=1 \mid x_2=1, x_5=1) = b_3 a_3 + b_4 a_4$

Eliminating X_5 replaces two tables by a single table over (X_2, X_3) , at a cost of $k^3 = 8$ products.

3.4 Conditioning to Marginalization Trick

A **notational trick** in which **conditioning is viewed as a summation**. It lets us treat marginalization and conditioning as **formally equivalent**, which brings the key operations of the inference algorithms into focus.

Definition : Evidence Potential

To capture the fact that X_i is fixed at the value $\bar{X}_i$, define the **evidence potential**

$$\delta(x_i, \bar{x}_i) = \begin{cases} 1 & \text{if } x_i = \bar{x}_i \\ 0 & \text{otherwise.} \end{cases}$$

It allows us to **turn evaluations into sums**:

$$g(\bar{x}_i) = \sum_{x_i} g(x_i) \delta(x_i, \bar{x}_i).$$

The trick also extends to **multivariate functions** with X_i as one of the arguments.

Proof: Evaluation as a Sum

Expanding the sum over all k values of x_i :

$$\sum_{x_i} g(x_i) \delta(x_i, \bar{x}_i) = g(0) \underbrace{\delta(0, \bar{x}_i)}_0 + \cdots + g(\bar{x}_i) \underbrace{\delta(\bar{x}_i, \bar{x}_i)}_1 + \cdots + g(k) \underbrace{\delta(k, \bar{x}_i)}_0 = g(\bar{x}_i).$$

■

Worked Example: Conditioning Turned into Marginalization

Directed graph: $p(\bar{x}_6 \mid x_2, x_5)$ from the running example can be written as

$$m_6(x_2, x_5) = \sum_{x_6} p(x_6 \mid x_2, x_5) \delta(x_6, \bar{x}_6) = p(\bar{x}_6 \mid x_2, x_5).$$

Undirected graph: $\psi(x_2, x_5, \bar{x}_6)$ can be written as

$$m_6(x_2, x_5) = \sum_{x_6} \psi(x_2, x_5, x_6) \delta(x_6, \bar{x}_6) = \psi(x_2, x_5, \bar{x}_6).$$

In both cases, **we have turned conditioning into marginalization!**

Definition : Total Evidence Potential

For the set of nodes X_E to be conditioned on,

$$\delta(x_E, \bar{x}_E) = \prod_{i \in E} \delta(x_i, \bar{x}_i) = \begin{cases} 1 & \text{if } x_E = \bar{x}_E \\ 0 & \text{otherwise.} \end{cases}$$

Key Rule: Conditionals Entirely by Summation

Both the numerator and the denominator of $p(x_F \mid \bar{x}_E)$ are obtained by summation:

$$p(x_F \mid \bar{x}_E) = \frac{p(x_F, \bar{x}_E)}{p(\bar{x}_E)} = \frac{\sum_{x_E} p(x_F, x_E) \delta(x_E, \bar{x}_E)}{\sum_{x_F} \sum_{x_E} p(x_F, x_E) \delta(x_E, \bar{x}_E)}.$$

Note: evidence potentials are merely a piece of **formal trickery** that simplifies the description of inference algorithms. In practice we would **not** sum over a function known to be zero over most of the sample space; we would instead take “**slices**” of the appropriate probabilities or potentials, as in the table-slicing example above.

3.5 The Variable Elimination Algorithm (Directed Graphs)

The algorithm maintains an **active list** of factors. Each update removes every factor that references the variable being eliminated, multiplies them, sums out the variable, and places the new intermediate factor back on the list.

Algorithm: Variable Elimination (Jordan, 2002)

```

procedure ELIMINATE( $\mathcal{G}, E, F$ ) ▷ main steps
  INITIALIZE( $\mathcal{G}, F$ )
  EVIDENCE( $E$ )
  UPDATE( $\mathcal{G}$ )
  NORMALIZE( $F$ )
end procedure

procedure INITIALIZE( $\mathcal{G}, F$ ) ▷ 1: ordering; local conditionals on active list
  choose an ordering  $I$  such that  $F$  appears last

```

```

for each node  $X_i$  in  $\mathcal{V}$  do
  place  $p(x_i \mid x_{\pi_i})$  on the active list
end for
end procedure

procedure EVIDENCE( $E$ ) ▷ 2: evidence potentials on active list
  for each  $i$  in  $E$  do
    place  $\delta(x_i, \bar{x}_i)$  on the active list
  end for
end procedure

procedure UPDATE( $\mathcal{G}$ ) ▷ 3: marginalization; update active list
  for each  $i$  in  $I$  do
    find all potentials on the active list that reference  $x_i$  and remove them
    let  $\phi_i(x_{T_i})$  denote the product of these potentials
    let  $m_i(x_{S_i}) = \sum_{x_i} \phi_i(x_{T_i})$ 
    place  $m_i(x_{S_i})$  on the active list
  end for
end procedure

procedure NORMALIZE( $F$ ) ▷ 4: desired conditional probability
   $p(x_F \mid \bar{x}_E) \leftarrow \phi_F(x_F) / \sum_{x_F} \phi_F(x_F)$ 
end procedure

```

Here x_{π_i} are the parents of X_i , T_i indexes all variables in the removed factors (so $i \in T_i$), and $S_i = T_i \setminus \{i\}$.

Worked Example: Running the Algorithm on $p(x_1 \mid \bar{x}_6)$

Evidence node X_6 , query node X_1 , and elimination ordering $I = \{6, 5, 4, 3, 2, 1\}$, in which the query node appears **last**.

Step 1 (Initialize): active list = $\{p(x_1), p(x_2|x_1), p(x_3|x_1), p(x_4|x_2), p(x_5|x_3), p(x_6|x_2, x_5)\}$.

Step 2 (Evidence): append $\delta(x_6, \bar{x}_6)$.

Step 3 (Update):

i	ϕ_i	$m_i = \sum_{x_i} \phi_i$	Active list afterwards
6	$p(x_6 x_2, x_5) \delta(x_6, \bar{x}_6)$	$m_6(x_2, x_5)$	$p(x_1), p(x_2 x_1), p(x_3 x_1), p(x_4 x_2), p(x_5 x_3), m_6$
5	$p(x_5 x_3) m_6(x_2, x_5)$	$m_5(x_2, x_3)$	$p(x_1), p(x_2 x_1), p(x_3 x_1), p(x_4 x_2), m_5$
4	$p(x_4 x_2)$	$m_4(x_2) = 1$	$p(x_1), p(x_2 x_1), p(x_3 x_1), m_5$ (ignore m_4)
3	$p(x_3 x_1) m_5(x_2, x_3)$	$m_3(x_1, x_2)$	$p(x_1), p(x_2 x_1), m_3$
2	$p(x_2 x_1) m_3(x_1, x_2)$	$m_2(x_1)$	$p(x_1), m_2$
1	$p(x_1) m_2(x_1)$	$m_1 = \sum_{x_1} p(x_1) m_2(x_1)$	m_1

At $i = 1$, $\phi_1(x_1) = p(x_1) m_2(x_1) = p(x_1, \bar{x}_6)$ is the **unnormalized conditional probability** and $m_1 = p(\bar{x}_6)$ is the **normalization factor**.

Step 4 (Normalize):

$$p(x_1 \mid \bar{x}_6) = \frac{\phi_1(x_1)}{\sum_{x_1} \phi_1(x_1)}.$$

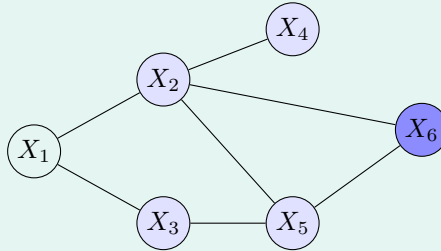
The same computation in algebraic form:

$$\begin{aligned}
p(\bar{x}_6) &= \sum_{x_1} \sum_{x_2} \sum_{x_3} \sum_{x_4} \sum_{x_5} \sum_{x_6} p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3)p(x_6|x_2, x_5) \delta(x_6, \bar{x}_6) \\
&\stackrel{i=6}{=} \sum_{x_1} \sum_{x_2} \sum_{x_3} \sum_{x_4} \sum_{x_5} p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2)p(x_5|x_3) m_6(x_2, x_5) \\
&\stackrel{i=5}{=} \sum_{x_1} \sum_{x_2} \sum_{x_3} \sum_{x_4} p(x_1)p(x_2|x_1)p(x_3|x_1)p(x_4|x_2) m_5(x_2, x_3) \\
&\stackrel{i=4}{=} \sum_{x_1} \sum_{x_2} \sum_{x_3} p(x_1)p(x_2|x_1)p(x_3|x_1) m_5(x_2, x_3) \quad (m_4(x_2) = 1) \\
&\stackrel{i=3}{=} \sum_{x_1} \sum_{x_2} p(x_1)p(x_2|x_1) m_3(x_1, x_2) \\
&\stackrel{i=2}{=} \sum_{x_1} p(x_1) m_2(x_1) \quad (\text{normalization factor})
\end{aligned}$$

where the summand of the last line is the unnormalized conditional $p(x_1, \bar{x}_6) = p(x_1) m_2(x_1)$.

3.6 Variable Elimination for Undirected Graphs**Key Rule: Only the Initialization Changes**

The entire variable elimination algorithm for directed graphs goes through **without essential change** to the undirected case. The only change is in INITIALIZE: instead of the local conditional probabilities, we initialize the active list with the **clique potentials** $\{\psi_{X_C}(x_C)\}$.

Worked Example: Elimination on an Undirected Graph

With $p(x) = \frac{1}{Z} \psi(x_1, x_2) \psi(x_1, x_3) \psi(x_2, x_4) \psi(x_3, x_5) \psi(x_2, x_5, x_6)$, evidence $\bar{x}_6$, ordering $(6, 5, 4, 3, 2, 1)$, and the shorthand $\psi_{ij} \equiv \psi(x_i, x_j)$, $\psi_{256} \equiv \psi(x_2, x_5, x_6)$:

$$\begin{aligned}
 p(x_1, \bar{x}_6) &= \frac{1}{Z} \sum_{x_2} \sum_{x_3} \sum_{x_4} \sum_{x_5} \sum_{x_6} \psi_{12} \psi_{13} \psi_{24} \psi_{35} \psi_{256} \delta(x_6, \bar{x}_6) \\
 &= \frac{1}{Z} \sum_{x_2} \psi_{12} \sum_{x_3} \psi_{13} \sum_{x_4} \psi_{24} \sum_{x_5} \psi_{35} \sum_{x_6} \psi_{256} \delta(x_6, \bar{x}_6) \\
 &= \frac{1}{Z} \sum_{x_2} \psi_{12} \sum_{x_3} \psi_{13} \sum_{x_4} \psi_{24} \sum_{x_5} \psi_{35} m_6(x_2, x_5) \\
 &= \frac{1}{Z} \sum_{x_2} \psi_{12} \sum_{x_3} \psi_{13} m_5(x_2, x_3) \sum_{x_4} \psi_{24} \\
 &= \frac{1}{Z} \sum_{x_2} \psi_{12} m_4(x_2) \sum_{x_3} \psi_{13} m_5(x_2, x_3) \\
 &= \frac{1}{Z} \sum_{x_2} \psi_{12} m_4(x_2) m_3(x_1, x_2) \\
 &= \frac{1}{Z} m_2(x_1).
 \end{aligned}$$

Unlike the directed case, $m_4(x_2) = \sum_{x_4} \psi(x_2, x_4)$ is in general **not** equal to one, since potentials are unnormalized. Marginalizing further over X_1 , and forming the conditional:

$$p(\bar{x}_6) = \frac{1}{Z} \sum_{x_1} m_2(x_1), \quad p(x_1 | \bar{x}_6) = \frac{m_2(x_1)}{\sum_{x_1} m_2(x_1)},$$

where the normalization factor Z **cancels**. (Image source: M. I. Jordan, 2002.)

Key Rule: When Does Z Cancel?

For a **conditional** probability, Z cancels between numerator and denominator. For a **marginal** probability, the normalization factor Z **does not cancel** and must be **calculated explicitly**.

4 Reconstituted Graph and Computational Complexity

Definition : Graph Elimination and the Reconstituted Graph

The variable elimination algorithm **successively eliminates** the nodes of $\mathcal{G}$ in the ordering I .

- “**Eliminate**” means **removing the node** from the graph and **connecting its (remaining) neighbors**. The new edges mirror the new intermediate factors: summing out x_i creates $m_i(x_{S_i})$, which couples all remaining neighbors of X_i .
- The **original and newly created edges** are recorded in the **reconstituted graph** $\tilde{\mathcal{G}} = (\mathcal{V}, \tilde{\mathcal{E}})$.

Algorithm: Undirected Graph Elimination (Jordan, 2002)

A simple **greedy** algorithm; the additional edges it adds form the reconstituted graph.

```

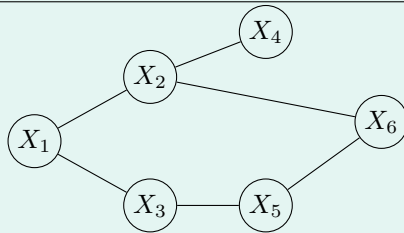
procedure UNDIRECTEDGRAPHELIMINATE( $\mathcal{G}, I$ )
  for each node  $X_i$  in  $I$  do
    connect all of the remaining neighbors of  $X_i$ 
    remove  $X_i$  from the graph
  end for
end procedure

```

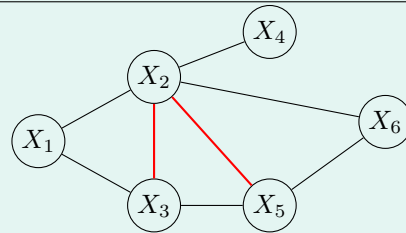
Worked Example: Reconstituted Graph for Ordering (6, 5, 4, 3, 2, 1)

Take $p(x) \propto \psi(x_1, x_2)\psi(x_1, x_3)\psi(x_2, x_4)\psi(x_3, x_5)\psi(x_2, x_6)\psi(x_5, x_6)$:

Eliminate	Remaining neighbors	New edge	Intermediate factor
X_6	X_2, X_5	$X_2 - X_5$	$m_6(x_2, x_5) = \sum_{x_6} \psi(x_2, x_6)\psi(x_5, x_6)$
X_5	X_2, X_3	$X_2 - X_3$	$m_5(x_2, x_3)$
X_4	X_2	—	$m_4(x_2)$
X_3	X_1, X_2	— (already present)	$m_3(x_1, x_2)$
X_2	X_1	—	$m_2(x_1)$



Original undirected graph



Reconstituted graph: additional edges (red) added during elimination

(Image source: M. I. Jordan, 2002.)

Definition : Elimination Clique and Treewidth

- The elimination process adds new edges between the (remaining) neighbors of a node, creating new **elimination cliques** $T_i = \{i\} \cup S_i$ in the graph.
- Treewidth**: one less than the **smallest achievable cardinality of the largest elimination clique**, over **all possible elimination orderings**.

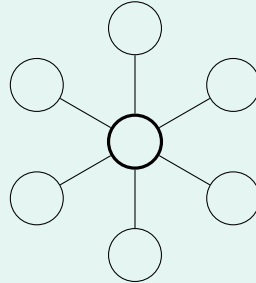
Key Rule: Complexity Is Governed by the Largest Elimination Clique

- Computing m_i sums over a table of size $k^{|T_i|}$, so the **overall complexity** is $O(k^r)$ with $r = \max_i |T_i|$, the size of the **largest elimination clique**.
- This depends on the **choice of elimination ordering**. The ordering with the **lowest complexity** must achieve the treewidth of the graph.
- Unfortunately, finding the best elimination ordering that achieves the treewidth is **NP-hard** in general, so heuristic orderings are used in practice.

In the example above the elimination cliques are $\{2, 5, 6\}$, $\{2, 3, 5\}$, $\{2, 4\}$, $\{1, 2, 3\}$, $\{1, 2\}$, $\{1\}$,

so $r = 3$ and elimination costs $O(k^3)$ rather than $O(k^6)$.

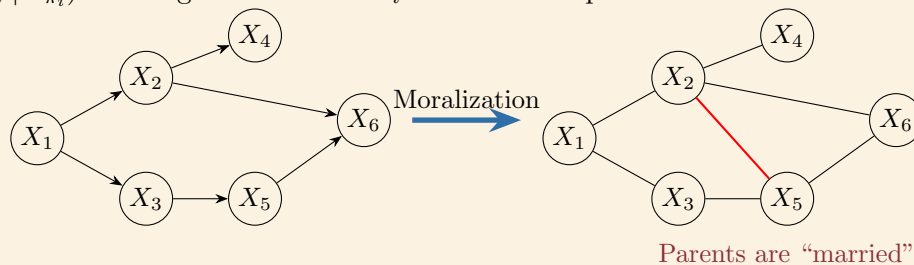
Worked Example: The Ordering Matters (Star Graph)



- This graph has **treewidth one**: eliminating the leaves first only ever creates cliques of size two.
- However, the wrong choice of **eliminating the center node first** immediately connects all six neighbors, giving an elimination clique that contains **every** node (size 7, i.e. $O(k^7)$).

Key Rule: Directed Graphs – Analyze via Moralization

The computational complexity of DGMs can be analyzed in the same way as UGMs by **moralization**: the DGM is converted into a UGM (parents are “married”, edge orientations dropped), where the complexity can be analyzed. Parents must be connected because each local conditional $p(x_i | x_{\pi_i})$ is a single factor over X_i and **all** of its parents.



Algorithm: Directed Graph Elimination (Jordan, 2002)

```

procedure DIRECTEDGRAPHELIMINATE( $\mathcal{G}, I$ )
   $\mathcal{G}^m \leftarrow \text{MORALIZE}(\mathcal{G})$ 
   $\text{UNDIRECTEDGRAPHELIMINATE}(\mathcal{G}^m, I)$ 
end procedure

procedure MORALIZE( $\mathcal{G}$ )
  for each node  $X_i$  in  $I$  do
    connect all of the parents of  $X_i$ 
  end for
  drop the orientation of all edges
  return  $\mathcal{G}$ 
end procedure

```

5 From Variable Elimination to Belief Propagation

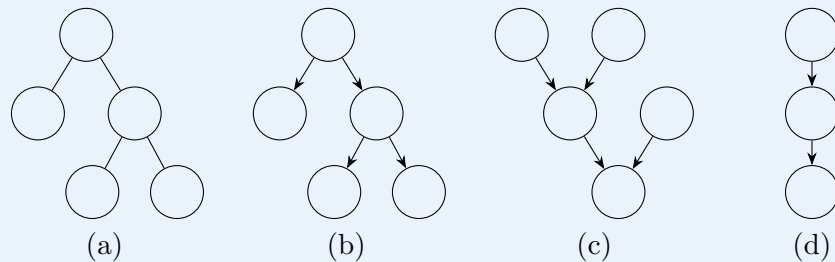
Key Rule: Limitation of Variable Elimination

Limitation: we have to **re-run** the variable elimination algorithm with **every new query node**.

Solution: the **sum-product** or **belief propagation** algorithm allows us to compute **all single-node marginals** for certain “**tree-like**” graphs in a **single run**.

5.1 Tree-Like Graphs

Definition : “Tree-Like” Graphs



(a) **Undirected tree:** without any loop.

(b) **Directed tree:** only one parent for every node; moralization leads to an undirected tree.

(c) **Polytree:** nodes with more than one parent. Not a directed tree; moralization leads to loops.

(d) **Chain:** also a directed tree (more on chains when we look at Hidden Markov Models).

(Image source: M. I. Jordan, 2002.)

5.2 Parameterization and Conditioning on Trees

Definition : Parameterization of Trees

Undirected trees: the cliques are **single nodes** and **pairs of nodes**, so for a tree $\mathcal{T}(\mathcal{V}, \mathcal{E})$

$$p(x) = \frac{1}{Z} \left(\prod_{i \in \mathcal{V}} \psi(x_i) \prod_{(i,j) \in \mathcal{E}} \psi(x_i, x_j) \right).$$

Directed trees: the joint probability is

$$p(x) = p(x_r) \prod_{(i,j) \in \mathcal{E}} p(x_j | x_i),$$

where $p(x_r)$ is the **marginal probability at the root**, $\{p(x_j | x_i)\}$ are the **conditional probabilities** at all other nodes, and (i, j) is a directed edge with i the parent of j .

Key Rule: Directed Tree $\rightarrow$ Undirected Tree

Define

$$\psi(x_r) = p(x_r), \quad \psi(x_i, x_j) = p(x_j \mid x_i) \quad (i \text{ the parent of } j),$$

and all other singleton potentials $\psi(x_i) = 1$ for all $i \neq r$. The partition function is then $Z = 1$.

We need not make any distinction between directed and undirected trees, since they are formally identical!

Definition : Evidence-Augmented Local Potentials

To capture conditioning, i.e. $p(x \mid \bar{x}_E)$ for some subset E , define the local potentials

$$\psi_i^E(x_i) \triangleq \begin{cases} \psi_i(x_i) \delta(x_i, \bar{x}_i) & i \in E \\ \psi_i(x_i) & i \notin E, \end{cases}$$

where $\delta(x_i, \bar{x}_i)$ is the evidence potential defined earlier. Using the joint of undirected trees, the conditional probability is

$$p(x \mid \bar{x}_E) = \frac{1}{Z^E} \left(\prod_{i \in \mathcal{V}} \psi_i^E(x_i) \prod_{(i,j) \in \mathcal{E}} \psi(x_i, x_j) \right), \quad Z^E = \sum_x \left(\prod_{i \in \mathcal{V}} \psi_i^E(x_i) \prod_{(i,j) \in \mathcal{E}} \psi(x_i, x_j) \right),$$

where the **original Z vanishes**. Conditioning therefore leaves us with another tree-structured distribution.

5.3 Messages

Question: what is special about variable elimination when the graph is a tree? **Answer:** we can use elimination orderings that arise from a **depth-first traversal** of the tree.

Key Rule: Depth-First Elimination on a Tree

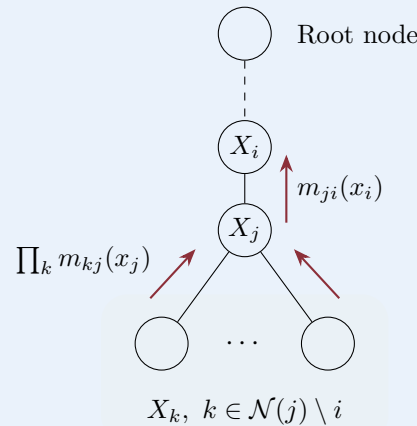
- Take advantage of the **recursive structure** of a tree to specify an elimination ordering.
- Treat the query node X_f as the **root**, and view the tree as directed with all edges pointing **away** from X_f .
- Elimination proceeds **inward from the leaves**. A node being eliminated has only one remaining neighbor (its parent), so no edges are added: the **treewidth equals one**.

Definition : Message $m_{ji}(x_i)$

Consider neighbors X_i and X_j in the tree, where X_i is **closer to the root**. To **eliminate** X_j , take the **product** over all potentials that reference X_j and **sum** over X_j :

$$m_{ji}(x_i) = \sum_{x_j} \left(\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}(x_j) \right)$$

This intermediate factor is the “**message**” that X_j sends to X_i . It depends only on x_i and summarizes all evidence in the subtree below X_j .



Key Rule: Marginal at the Final Node (Root)

All other nodes have been eliminated by the time we arrive at X_f , so messages $m_{ef}(x_f)$ have been computed for each neighbor $e \in \mathcal{N}(f)$. The marginal of X_f is

$$p(x_f \mid \bar{x}_E) \propto \psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}(x_f)$$

with normalizer $Z^E = \sum_{x_f} \psi^E(x_f) \prod_{e \in \mathcal{N}(f)} m_{ef}(x_f)$.

5.4 Reusing Messages

It turns out that these messages are sufficient for obtaining **not only a single marginal**, but **all of the marginals** in the tree.

Key Rule: Messages Can Be “Reused”!

A message m_{ji} depends only on the subtree on X_j 's side of the edge (i, j) , **not** on which node is the root. Hence we can achieve the effect of computing over **all possible elimination orderings** (a huge number) by computing **all possible messages** (a small number: one per direction of each edge, i.e. $2|\mathcal{E}|$).

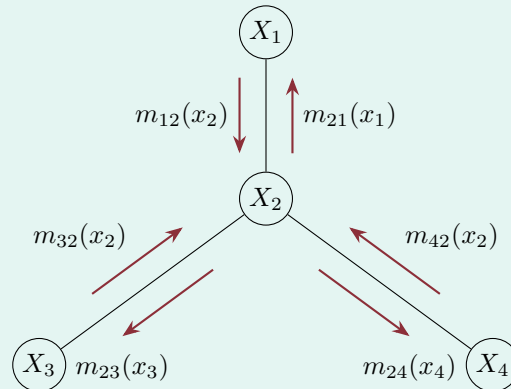
Worked Example: Four-Node Tree, Four Queries

Consider the tree with edges $X_1 - X_2$, $X_2 - X_3$, $X_2 - X_4$. Running elimination with each node in turn as the query node requires:

Query node	Messages needed	Marginal
X_1	$m_{32}(x_2)$, $m_{42}(x_2)$, $m_{21}(x_1)$	$\propto \psi^E(x_1) m_{21}(x_1)$
X_2	$m_{12}(x_2)$, $m_{32}(x_2)$, $m_{42}(x_2)$	$\propto \psi^E(x_2) m_{12} m_{32} m_{42}$
X_3	$m_{12}(x_2)$, $m_{42}(x_2)$, $m_{23}(x_3)$	$\propto \psi^E(x_3) m_{23}(x_3)$
X_4	$m_{12}(x_2)$, $m_{32}(x_2)$, $m_{24}(x_4)$	$\propto \psi^E(x_4) m_{24}(x_4)$

The colored messages are **reused** across queries. All six messages below suffice to compute

every singleton marginal; the **sum-product algorithm** computes exactly these messages efficiently.



(Image source: M. I. Jordan, 2002.)

6 The Sum-Product Algorithm

Key Rule: Message-Passing Protocol

A node can send a message to a neighboring node **when (and only when)** it has received messages from **all of its other neighbors**.

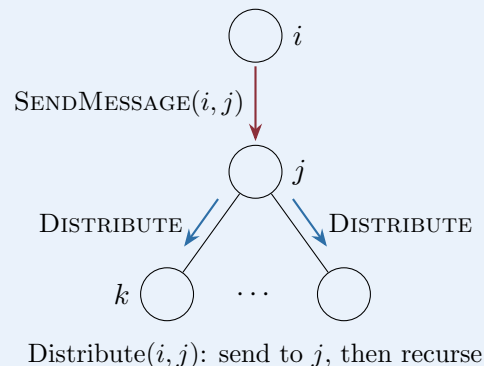
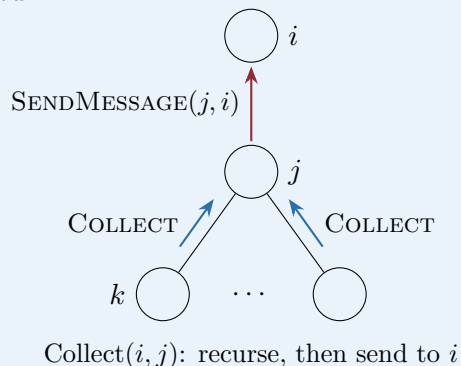
This rule follows directly from the message definition: m_{ji} needs every m_{kj} with $k \in \mathcal{N}(j) \setminus i$.

Definition : The Two Phases of Sum-Product

Pick any node as the root, then:

1. **Collect (inward):** messages flow from the **leaves toward the root**.
2. **Distribute (outward):** initiated once **all incoming messages have been received by the root**; messages flow from the **root toward the leaves**.

After both phases every node has heard from all of its neighbors, so its marginal can be computed.



Algorithm: Sum-Product on a Tree (Jordan, 2002)

```

procedure SUM-PRODUCT( $\mathcal{T}, E$ ) ▷ main steps
  EVIDENCE( $E$ )
   $f \leftarrow \text{CHOOSEROOT}(\mathcal{V})$ 
  for  $e \in \mathcal{N}(f)$  do
    COLLECT( $f, e$ )
  end for
  for  $e \in \mathcal{N}(f)$  do
    DISTRIBUTE( $f, e$ )
  end for
  for  $i \in \mathcal{V}$  do
    COMPUTEMARGINAL( $i$ )
  end for
end procedure

procedure EVIDENCE( $E$ ) ▷ conditioning  $\rightarrow$  marginalization
  for  $i \in E$  do
     $\psi^E(x_i) = \psi(x_i) \delta(x_i, \bar{x}_i)$ 
  end for
  for  $i \notin E$  do
     $\psi^E(x_i) = \psi(x_i)$ 
  end for
end procedure

procedure COLLECT( $i, j$ ) ▷ inward: leaves  $\rightarrow$  root
  for  $k \in \mathcal{N}(j) \setminus i$  do
    COLLECT( $j, k$ )
  end for
  SENDMESSAGE( $j, i$ )
end procedure

procedure DISTRIBUTE( $i, j$ ) ▷ outward: root  $\rightarrow$  leaves
  SENDMESSAGE( $i, j$ )
  for  $k \in \mathcal{N}(j) \setminus i$  do
    DISTRIBUTE( $j, k$ )
  end for
end procedure

procedure SENDMESSAGE( $j, i$ ) ▷ intermediate factor (message)
   $m_{ji}(x_i) = \sum_{x_j} (\psi^E(x_j) \psi(x_i, x_j) \prod_{k \in \mathcal{N}(j) \setminus i} m_{kj}(x_j))$ 
end procedure

procedure COMPUTEMARGINAL( $i$ ) ▷ message to final node
   $p(x_i) \propto \psi^E(x_i) \prod_{j \in \mathcal{N}(i)} m_{ji}(x_i)$ 
end procedure

```

Key Rule: Cost of Sum-Product

Each message is a sum over x_j for every value of x_i , i.e. $O(k^2)$ once its incoming messages are available. With $2|\mathcal{E}|$ messages, sum-product computes **all** singleton marginals of a tree in $O(|\mathcal{E}|k^2)$ – roughly twice the cost of a single elimination run, instead of $|\mathcal{V}|$ separate runs. (*Scribe's note; not stated on the slides.*)

Worked Example: Sum-Product with Numbers (Scribe's Example)

This example is not from the slides; it is included to illustrate the computation. Use the four-node tree above with $x_i \in \{0, 1\}$, uniform singleton potentials $\psi(x_i) = 1$, and the same “agreement” potential on every edge

$$\psi(x_i, x_j) = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \quad (\text{rows } x_i, \text{ columns } x_j).$$

We observe $\bar{x}_4 = 1$, so $\psi^E(x_4) = \delta(x_4, 1) = (0, 1)$ and $\psi^E(x_i) = (1, 1)$ for $i \neq 4$. Choose X_1 as the root; vectors list values at $x = 0$ and $x = 1$.

Collect (toward X_1):

$$\begin{aligned} m_{32}(x_2) &= \sum_{x_3} \psi^E(x_3) \psi(x_2, x_3) = (2 + 1, 1 + 2) = (3, 3) \\ m_{42}(x_2) &= \sum_{x_4} \delta(x_4, 1) \psi(x_2, x_4) = \psi(x_2, 1) = (1, 2) \\ m_{21}(x_1) &= \sum_{x_2} \psi(x_1, x_2) m_{32}(x_2) m_{42}(x_2) = \sum_{x_2} \psi(x_1, x_2) (3, 6) \\ &= (2 \cdot 3 + 1 \cdot 6, 1 \cdot 3 + 2 \cdot 6) = (12, 15) \end{aligned}$$

Distribute (away from X_1):

$$\begin{aligned} m_{12}(x_2) &= \sum_{x_1} \psi^E(x_1) \psi(x_1, x_2) = (3, 3) \\ m_{23}(x_3) &= \sum_{x_2} \psi(x_2, x_3) m_{12}(x_2) m_{42}(x_2) = \sum_{x_2} \psi(x_2, x_3) (3, 6) = (12, 15) \\ m_{24}(x_4) &= \sum_{x_2} \psi(x_2, x_4) m_{12}(x_2) m_{32}(x_2) = \sum_{x_2} \psi(x_2, x_4) (9, 9) = (27, 27) \end{aligned}$$

Marginals:

Node	Unnormalized	$p(x_i \mid \bar{x}_4)$
X_1	$m_{21}(x_1) = (12, 15)$	$(4/9, 5/9)$
X_2	$m_{12} m_{32} m_{42} = (9, 18)$	$(1/3, 2/3)$
X_3	$m_{23}(x_3) = (12, 15)$	$(4/9, 5/9)$
X_4	$\delta(x_4, 1) m_{24}(x_4) = (0, 27)$	$(0, 1)$

Every unnormalized marginal sums to the same $Z^E = 27$, as it must. Observing $X_4 = 1$ makes $X_2 = 1$ more likely, and the effect weakens with distance (X_1, X_3). **Brute-force check:** $p(x \mid \bar{x}_4) \propto \psi(x_1, x_2) \psi(x_2, x_3) \psi(x_2, 1)$; summing out x_1, x_3 gives $3 \cdot 3 \cdot \psi(x_2, 1) = (9, 18)$, matching the table.

Summary: Variable Elimination vs. Sum-Product

	Variable Elimination	Sum-Product (BP)
Graphs	any DGM / UGM	trees (tree-like graphs)
Output per run	one marginal / conditional	all singleton marginals
Intermediate factors	$m_i(x_{S_i})$, discarded after use	messages $m_{ji}(x_i)$, reused
Cost	$O(k^r)$, r = largest elimination clique	$O(\mathcal{E} k^2)$ (treewidth one)
Key difficulty	choosing a good ordering (NP-hard)	message scheduling (protocol)

Take-away: sum-product *is* variable elimination on a tree, organized so that each intermediate factor is computed once and shared by every query.

7 Summary

- Used the Variable Elimination algorithm to compute the conditional probability of a single random variable X_f , i.e. $p(x_f | x_E)$.
- Explained the computational complexity of variable elimination using the reconstituted graph.
- Used the sum-product algorithm to compute all single-node marginals for “tree-like” graphical models.

Key Takeaways

1. **Inference** means computing $p(x_F | \bar{x}_E)$ by summing out the nuisance variables X_R ; naive summation over the joint costs $O(k^n)$.
2. **Variable elimination** writes the joint in factored form and uses the **distributive law** to sum out one variable at a time, producing intermediate factors $m_i(x_{S_i})$. For DGMs the active list starts with the local conditionals; for UGMs, with the clique potentials.
3. **Evidence potentials** $\delta(x_i, \bar{x}_i)$ turn conditioning into marginalization, so numerator and denominator of a conditional are both sums. In practice we simply take slices of tables. For UGMs, Z cancels in conditionals but **not** in marginals.
4. **Complexity** is set by the **largest elimination clique**, read off the **reconstituted graph**; it depends on the elimination ordering, the best achievable is governed by the **treewidth**, and finding the optimal ordering is **NP-hard**. DGMs are analyzed after **moralization**.
5. On **trees**, depth-first elimination has treewidth one, and intermediate factors become **messages** $m_{ji}(x_i)$ that can be **reused** across queries.
6. The **sum-product (belief propagation)** algorithm obeys the **message-passing protocol** with a **Collect** (inward) and a **Distribute** (outward) phase, computing all $2|\mathcal{E}|$ messages and hence **all** singleton marginals $p(x_i | \bar{x}_E) \propto \psi^E(x_i) \prod_{j \in \mathcal{N}(i)} m_{ji}(x_i)$ in a single run.

Acknowledgement: Slide content adapted from M. I. Jordan, “An Introduction to Probabilistic Graphical Models” (2002, Chapters 3 and 4.1); K. Murphy, “Machine Learning: A Probabilistic Perspective” (Ch. 20.1–20.3); D. Koller & N. Friedman, “Probabilistic Graphical Models” (Ch. 9); D. Barber, “Bayesian Reasoning and Machine Learning” (Ch. 5); C. Bishop, “Pattern Recognition and Machine Learning” (Ch. 8, Section 8.4).